CORE JAVA SERIES — DAY 10

Nested Classes & Standard Input/Output

The four kinds of nested classes — static nested, inner, local, and anonymous — with the effectively-final rule and memory internals; then console I/O end-to-end: deconstructing `System.out.println`, byte streams and `InputStream`, the Reader hierarchy, `BufferedReader`, and the `Scanner` class.

Nested Classes — Concept & Motivation

Just as loops and switch statements can nest, **a class can be declared inside another class**. The enclosing class is the **outer** class; the nested one is the **inner** class. Nesting can go arbitrarily deep (A → B → C → D), but two levels suffice in practice. Java defines **four kinds** of nested classes:

Kind	Declared	Has a name?
1. Static nested class	Inside a class, marked `static`	Yes
2. Inner class	Inside a class, **no** `static` keyword	Yes
3. Local class	Inside *any code block* (method, if, loop, static block…)	Yes
4. Anonymous class	At an object-creation site, nameless	**No**

> **Why nest at all?** (1) **Logical grouping** — some classes only make sense inside another class; existing standalone, they would "sound wrong." (2) **Better access** — a nested class can touch the outer class's **private** members directly, with no getters/setters, because it lives inside the outer class's own body.

1. Static Nested Class

```
class Outer {
    static int x = 4;
    int y;                                  // non-static member

    static class Inner {
        void fun() {
            System.out.println(x);          // OK — static member
            // System.out.println(y);  ✗ "cannot make a static reference
                                       //     to the non-static field y"
        }
    }
}
```

Object creation

```
Outer.Inner inner = new Outer.Inner();   // NO Outer object needed
inner.fun();
```

The nested class's **qualified name is `Outer.Inner`** — the JVM locates it as *package → Outer → Inner*. Because the class is `static`, it belongs to `Outer` the **class**, not to any object — so no `Outer` instance is required, exactly like accessing a static field via the class name. Consequently, it may touch only the outer class's **static** members; a non-static member like `y` belongs to an object that may not exist. (Indirect access is possible: accept an `Outer` reference as a parameter or constructor argument and use `outer.y`.)

Otherwise it behaves like a completely normal class: it may extend a class, implement interfaces, declare any number of methods/variables, have constructors — and, unlike inner classes, it **may declare static members** (static methods, variables, blocks). Access modifiers work too — including making the *entire nested class* `private` (see the BankAccount example below).

Killer use case — a hidden helper class

```
class BankAccount {
    public double computeInterest(double principal) {
        return InterestCalculator.calculateYearly(principal, 0.09);
    }
    private static class InterestCalculator {         // invisible outside!
        static double calculateYearly(double p, double rate) { return p * rate; }
    }
}
```

`InterestCalculator` is a helper that only makes sense for `BankAccount` — so we nest it, and declaring it `private` **hides a whole class** from the outside world; only `BankAccount`'s own methods can use it. **Other use cases:** the Builder design pattern, DTOs/request-response objects in Spring Boot.

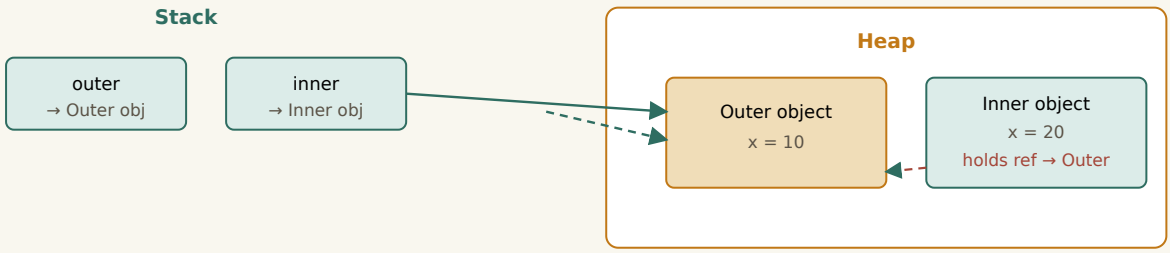
2. Inner Class (non-static nested)

```
class Outer {
    int x = 10;
    class Inner {
        int x = 20;
        void fun() {
            System.out.println(x);              // 20 — inner's field shadows
            System.out.println(Outer.this.x);   // 10 — outer's field
        }
    }
}
```

Object creation — requires an Outer object

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();   // or one-liner:
Outer.Inner inner2 = new Outer().new Inner();
```

Without `static`, the inner class belongs to an **instance** of the outer class — so an `Outer` object must exist first. The dot-operator logic makes the syntax intuitive: `outer.new Inner()` means "go inside this outer object, then create an Inner."



Each inner object is a separate heap block that secretly stores a reference to its enclosing outer object.

The hidden outer reference. Every non-static inner object carries an implicit reference to the outer object that created it (the compiler inserts it — you never write it). This explains everything: (1) the inner class can freely use **all** outer members — static or not, private or public; (2) each outer object gets its *own* inner objects; (3) to reach a shadowed outer field, use `Outer.this.x` — otherwise the inner field wins.

The static-member restriction (pre-Java 16)

Old Java forbade static members inside an inner class. First principles: `static` means "one copy per **class**, shared by all objects" — but the inner class *itself* is per-outer-*object*. So a static `x` inside Inner creates an unresolvable ambiguity: should `x` be one copy for all inner objects (static semantics), or one copy *per outer object* (the class it lives in)? Rather than resolve the confusion, Java banned it outright — use a **static nested** class if you need static members. (Java 16+ relaxed this; the modern reasoning is covered with newer-version features.)

Active Recall — Part 1

Q1. How do you instantiate `Outer.Inner` when Inner is static vs non-static?

A. Static: `new Outer.Inner()` — no outer object. Non-static: `outer.new Inner()` — an outer object must exist first.

Q2. Why can't a static nested class touch non-static outer members directly?

A. It may exist with zero outer objects; a non-static member belongs to an object that might not exist ("static reference to non-static field").

Q3. How does an inner object reach an outer field that it has shadowed?

A. `Outer.this.fieldName`.

Q4. What was the ambiguity that banned static members in inner classes?

A. Static = one copy per class, but an inner class is per-outer-object — so "one shared copy" vs "one per outer object" couldn't both be satisfied.

3. Local Class

A class declared **inside any code block** — a method, constructor, `if`, loop, or even a static block:

```
class Outer {
    void greet() {
        int y = 5;                      // local variable
        class Local {
            void sayHello() { System.out.println(y); }
        }
        Local local = new Local();
        local.sayHello();
    }
}
```

Scope is the block itself — the class ceases to exist when the method returns, and nothing outside the block can even *name* it. All inner-class rules apply (full access to outer members), **plus one strict rule**:

> **The effectively-final rule.** A local class may read a local variable of the enclosing method **only if that variable is final or effectively final** — i.e. assigned exactly once and never modified (`y++` after the class → compile error: "*Local variable y defined in an enclosing scope must be final or effectively final*").
>
> **Why:** the method's stack frame (and its locals) vanish at return, but the local-class **object lives on in the heap** — e.g. if some `Object`-typed reference is returned. To keep the variable usable, the **compiler silently copies** its value into the class as an instance field. If the original could change after the copy, the JVM would face an ambiguity: *which* value should have been copied? Banning mutation removes the ambiguity entirely.

Verdict: legitimate but rare — almost any local class can be replaced by simpler code; you may never write one in production.

4. Anonymous Class

An anonymous class is **a class without a name**, created for a job needed exactly once. Instead of writing a whole `Guest extends Person` file to override one method for a single call site:

```
Person p2 = new Person() {           // semicolon replaced by a body ⇒
    @Override                        // anonymous class begins here
    void introduce() { System.out.println("Hi I am a guest"); }
    String name = "Aditya";           // own fields allowed
    void greet() { System.out.println("hello"); }   // own methods allowed…
};                                    // …but see below
```

- **Rules:** it behaves like an inner class (full outer access; if declared inside a method, the effectively-final rule applies too). Fields and extra methods **are** allowed.
- **No constructors** — a constructor must share the class's name, and this class has no name. (The `Person()` call invokes the *parent's* constructor.)
- **The reference-type trap:** `p2` is a `Person` reference, so only `Person` members are visible through it — `p2.greet()` or `p2.name` won't compile. Extra members exist solely to be called *from inside* the overridden method.
- **Use it when** a one-off, localized override is needed; if the behavior is needed repeatedly, write a real named class instead.

Frequency in modern codebases: static nested ≫ inner ≫ anonymous ≫ local. Anonymous classes were everywhere in old Java but have been largely **replaced by lambdas** (coming lectures). Local classes are the rarest of all.

Active Recall — Part 2

Q1. Where can a local class be declared, and what is its scope?

A. Inside any code block; it is visible only within that block and dies with it.

Q2. State the effectively-final rule and its reason.

A. A local class may only read enclosing locals that are never reassigned — because the compiler copies the value into the object, and a later change would make the copied value ambiguous.

Q3. Why can an anonymous class have no constructor?

A. A constructor must be named after its class, and an anonymous class has no name.

Q4. You added a helper method inside an anonymous class — why can't you call it via the reference?

A. The reference is typed as the parent class, which doesn't declare that method; extra members are reachable only from inside the class body.

Standard Input/Output — Console I/O

So far every program ran on data already inside it. Real programs must communicate: **console** I/O (keyboard/screen), **file** I/O, **network** I/O (API calls — Spring Boot), and **memory-buffer** I/O. This lecture covers the console; file and network I/O come later.

Deconstructing `System.out.println`

```
System.out.println("hello");
// │     │     └─ println(String) — the method that prints + newline
// │     └─ out: a static reference variable of type PrintStream
// └─ System: a class in java.lang (auto-imported)
```

- `System` is a class in the **`java.lang` package — automatically imported** into every file, which is why no import is ever written.
- Inside `System`: `public static final PrintStream out = …;` — `static` so we reach it by class name, `final` so it can't be reassigned.
- `PrintStream` is a concrete class (a subclass in the `OutputStream` hierarchy) whose methods print to the **console**: `println` (with newline), `print` (no newline), `printf` (formatted output).

> **`System.err` — the twin stream.** Also a `public static final PrintStream`, differing only in name. Convention: normal output via `out`, **errors** via `err` (`System.err.println("Invalid age")`). This separation lets you (and the OS/log tooling) route error messages differently — e.g. show them in red — and it works even in modern logging setups.

Streams — the Foundation of Java I/O

A **stream is a flow of data** — picture a pipe carrying data. Java's entire I/O is **stream-based**:

Abstract class	Data flows	Core method
`InputStream`	**into** the program (read)	`read()`
`OutputStream`	**out of** the program (write)	`write()`

Both are **abstract** — they merely declare `read`/`write` without saying *where from*. Concrete subclasses decide the source/sink: `FileInputStream`, `ByteArrayInputStream`, `BufferedInputStream`, `DataInputStream` (and their Output counterparts, with `PrintStream` as the console specialist). Crucially, these are **byte streams** — data flows as **one byte at a time**.

`System.in` and the primitive way to read

`System.in` is `public static final InputStream in` — attached **by default to the keyboard**. Its `read()` returns an `int`: type 'A' and you get back **65** — the ASCII value, arriving one byte at a time. To read "Aditya" you must call `read()` in a loop until `'\n'`, concatenating characters — clearly impractical:

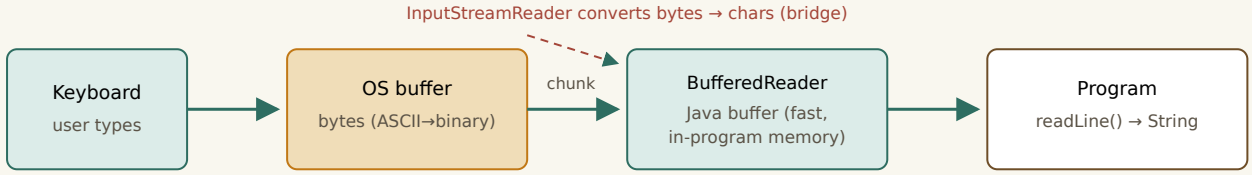
```
int c;
String s = "";
while ((c = System.in.read()) != '\n') {   // 1 byte per call!
    s += (char) c;
}
```

Typed input first lands in the OS input buffer; each `read()` fetches a single byte from it.

Readers — Characters Instead of Bytes

Java added a parallel hierarchy: the abstract `Reader` class, whose subclasses read **streams of characters** (no manual casting). Key concrete classes: `InputStreamReader`, `BufferedReader`, `FileReader`.

Problem	Solution class	What it does
`InputStream` (bytes) is incompatible with `BufferedReader` (characters)	`InputStreamReader`	**Converts** a byte stream into a character stream
One-byte-at-a-time reads mean an OS call per byte — slow	`BufferedReader`	Reads a **chunk** from the OS buffer in one go, stores it in its own in-program buffer; the program then reads from that fast local buffer



The full input pipeline: keyboard → OS buffer → InputStreamReader (byte→char) → BufferedReader (chunked local buffer) → program.

```
import java.io.*;
BufferedReader br = new BufferedReader(
        new InputStreamReader(System.in));   // the classic one-liner
String name = br.readLine();                // whole line as a String
```

BufferedReader's limitations: it reads only **Strings** — an int needs a second step: `int i = Integer.parseInt(br.readLine());` — and even the one-liner is ceremony-heavy. Java needed something simpler…

The Scanner Class (Java 5+)

`Scanner` lives in `java.util` (a **utility**, not part of the I/O stream hierarchies — it works *on top* of them, a wrapper). It fixed both limitations:

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);       // still an InputStream inside!
String s  = sc.nextLine();
int    i  = sc.nextInt();                  // no parsing needed
double d  = sc.nextDouble();
boolean b = sc.nextBoolean();
```

- **One object, one line** — the whole double-wrapping ceremony disappears.
- **Type-specific `nextX()` methods** for every primitive: `nextInt, nextDouble, nextFloat, nextLong, nextShort, nextByte, nextBoolean` — reading and conversion in one call.
- **Tokenization:** Scanner splits input on a delimiter (default: whitespace). `next()` returns **one token** ("Aditya Tandon" → "Aditya"); `nextLine()` returns the **entire line**.
- **Not just keyboard:** pass a `File` or even a `String` to the constructor to scan those instead.

> **Scanner is slower than BufferedReader.** Internally Scanner tokenizes, uses regular expressions, and performs type conversion; BufferedReader just moves characters. For most applications Scanner is the right choice (simplicity wins); reach for BufferedReader in competitive programming or when microseconds matter.

Active Recall — Part 3

Q1. What exactly are `out` and `err`, and why do both exist?

A. Both are `public static final PrintStream` fields of `System`; `out` is for normal output, `err` for error output so the two can be routed/handled differently.

Q2. Why are `InputStream` / `OutputStream` abstract, and what do they leave to subclasses?

A. They only declare read/write; the concrete subclass (File, Buffered, Print…) decides how reading/writing is implemented for its source/sink.

Q3. What two jobs do `InputStreamReader` and `BufferedReader` each perform?

A. InputStreamReader converts the byte stream to a character stream; BufferedReader reads chunks from the OS buffer into a fast in-program buffer.

Q4. `next()` vs `nextLine()`, and when is BufferedReader preferred over Scanner?

A. `next()` reads one whitespace-delimited token, `nextLine()` the whole line; BufferedReader wins when raw speed matters (competitive programming), since Scanner pays for tokenization/regex/conversion.